National University of Computer and Emerging Sciences Islamabad
Operating System Lab Spring 2025

Lab Task: Inter-Process Communication (IPC)

Task 1: Read and Write

Objective:

Write a program that reads user input from the keyboard using **read()** and prints it back to the screen using **write()**, without using standard I/O functions like printf() or scanf().

Instructions:

1. **Prompt the user** to enter a string.
2. **Read the input** from the keyboard using read().
3. **Write the input** back to the screen using write().
4. **Repeat the process** until the user enters "exit".
5. **Ensure proper error handling** for failed read/write operations.

Expected Behavior:

Input Example:

- Hello

Output Example:

- You entered: Hello

Exit Condition:
If the user types "exit", the program should terminate.

Task 2: Two-Way Parent-Child Communication Using Pipes

Objective:

- Implement **bidirectional communication** between a **parent and child process** using pipes.
- The **parent process** sends **three messages** to the child through a pipe.
- The **child reads each message**, prints it, and **sends a response** back to the parent via another pipe.
- Ensure **proper synchronization** so messages are received and acknowledged correctly.

Instructions:

1. Create **two pipes**: one for Parent → Child and another for Child → Parent.
2. Use `fork()` to create a child process.
3. The parent **writes three messages** to the child process.
4. The child **reads each message**, prints it, and sends a response.
5. The parent **reads acknowledgments** from the child and prints them.
6. Close unused ends of the pipes in both parent and child processes to avoid deadlocks.

Expected Input:

- Parent sends: "Message 1", "Message 2", "Message 3" to the child.

Expected Output:

- **Child prints:** "Received Message 1", "Received Message 2", "Received Message 3".
- **Child sends acknowledgments:** "Acknowledged Message 1", "Acknowledged Message 2", "Acknowledged Message 3".
- **Parent prints child's acknowledgments**.

Task 3: Multi-Child Computation Using Pipes

Objective:

- Implement a **parent process** that creates **three child processes**.
- Each child should perform a **different computation** and send its **result** back to the parent via pipes.
- The parent should **collect results from all child processes** and print them.

Computations Performed by Each Child:

- **Child 1:** Computes the **factorial** of a given number.
- **Child 2:** Computes the **sum of numbers up to N**.
- **Child 3:** Computes the **cube** of a given number.

Instructions:

1. Create **three pipes** for communication between the parent and each child.
2. Use `fork()` three times to create three child processes.
3. Each child process performs its assigned computation and **writes the result** to its respective pipe.
4. The parent process **reads results from all pipes** and displays them.
5. Ensure proper **closing of unused pipe ends** to prevent deadlocks.

Expected Input:

- A number `N` provided as input to the program.

Expected Output:

- **Child 1 computes:** Factorial of `N` and sends it to the parent.
- **Child 2 computes:** Sum of numbers up to `N` and sends it to the parent.
- **Child 3 computes:** Cube of `N` and sends it to the parent.
- **Parent receives and prints all results.**

Instructions for Submission

- Ensure your code is **well-documented with comments**.
- Implement **proper error handling** (e.g., file not found, read/write failure, pipe issues).
- Test your program with **various inputs** before submission.
- Submit your **source code along with screenshots of successful runs**.

THE END
